Final Project Report

ATPG Using D-Algorithm, PODEM, and SAT with Z3 Solver

Ahmed Ghoneim And Samir Ahmed

This project implements a complete Automatic Test Pattern Generation (ATPG) environment capable of parsing combinational logic circuits, performing stuck-at fault collapsing, simulating good and faulty circuits under Roth's five-valued logic, and generating detecting test patterns using the D-Algorithm, PODEM, and a SAT-based ATPG engine using the Z3 SMT solver. The tool is designed to work with a wide variety of .ckt formats, including ISCAS-style expressions and the textbook-style column-formatted netlists that are given to us through Canvas.

At the core of the implementation is a modular Python codebase. The circuit parser (ckt_parser.py) is intentionally flexible so that it can cleanly interpret primary-input/primary-output declarations, gate tables, and mixed formatting. It automatically constructs the circuit graph and topological ordering required for ATPG algorithms. The circuit simulation engine (circuit.py) implements Roth's five-valued logic (0, 1, X, D, D′), allowing consistent fault injection at a single node and propagation of that effect through combinational logic. Gate behaviors are defined in logic5.py, ensuring correctness for NAND, NOR, AND, OR, and NOT primitives under the five-valued algebra.

Three ATPG algorithms were developed and integrated. The D-Algorithm (d_algorithm.py) uses the classical approach of generating and propagating D/D′ symbols while maintaining consistency through justification steps and backtracking. PODEM (podem.py) improves search efficiency by restricting decisions to primary inputs and using a backtrace mechanism to guide assignments. Both algorithms use the same implication mechanism implemented by the circuit simulator. Complementing these structural ATPG engines is the SAT-based ATPG solver (sat_atpg.py), which constructs a Boolean model containing a good and faulty copy of the circuit, encodes the stuck-at behavior and required activation, ties primary inputs between copies, and asserts output mismatch. The resulting formula is solved using the Z3 SMT solver, and any satisfying assignment returned by Z3 corresponds to a valid ATPG test pattern.

Fault modeling is handled by fault.py, which generates stuck-at-0 and stuck-at-1 faults for every net. For these benchmarks, every fault remained testable after fault collapsing. The main program (atpg.py) provides an interactive user interface that allows netlist loading, simulation, listing collapsed faults, and generating ATPG patterns using any of the three implemented engines.

All algorithms were tested on five benchmark circuits: t4_3, t4_21, t5_10, t5_26a, and t6_24. Each benchmark was parsed correctly, fault lists were generated, tests were computed, and all results were verified via explicit five-valued simulation. The SAT engine always produced fully specified 0/1 patterns, while D-Algorithm and PODEM often returned partial tests with don't-care inputs (X), as expected for structural ATPG methods. In every circuit, the tests produced by structural methods corresponded to valid Boolean solutions—meaning that SAT patterns always satisfied at least one expansion of the D-Algorithm/PODEM test cubes.

Benchmark ATPG Results Summary

All results were obtained using real program output (see results log

results

).
Below is a clean formatted summary of fault counts and detection performance.

t4_3 : 9-node circuit

Total faults after collapsing: 18

D-Algorithm: detected 18/18 faults

PODEM: detected 18/18 faults

SAT (Z3): detected 18/18 faults

t4_21 : 10-node circuit

Total faults (collapsed): 20

D-Algorithm: 20/20

PODEM: 20/20

SAT: 20/20

t5_10 : textbook NAND tree

Total faults: 20

D-Algorithm: 20/20

PODEM: 20/20

SAT: 20/20

t5_26a : Full Adder

Total faults: 24

D-Algorithm: 24/24

PODEM: 24/24

SAT: 24/24

t6_24 : Large Multi-Level Circuit

Total collapsed faults: 44

D-Algorithm: 44/44

PODEM: 44/44

SAT: 44/44

SAT patterns were manually re-verified by explicit fault simulation, confirming correctness for all 44 faults.

Here are the FULL terminal results of our project:
t4_3.ckt
[0] Read the input net-list
[1] Perform fault collapsing
[2] List fault classes
[3] Simulate
[4] Generate tests (D-Algorithm)
[5] Generate tests (PODEM)

```
[6] Generate tests (Boolean Satisfaibility)
[7] Exit

Select an option: 0
Enter path to .ckt file: examples/t4_3.ckt
Loaded circuit with 9 nodes, 4 PIs, 1 POs.

[0] Read the input net-list
[1] Perform fault collapsing
[2] List fault classes
[3] Simulate
[4] Generate tests (D-Algorithm)
[5] Generate tests (PODEM)
[6] Generate tests (Boolean Satisfaibility)
[7] Exit

Select an option: 1
Created 18 fault classes

[0] Read the input net-list
[1] Perform fault collapsing
[2] List fault classes
[3] Simulate
[4] Generate tests (D-Algorithm)
[5] Generate tests (PODEM)
[6] Generate tests (Boolean Satisfaibility)
[7] Exit

Select an option: 2
Class 0: 1gat-sa0
Class 1: 1gat-sa1
Class 2: 2gat-sa0
Class 3: 2gat-sa1
Class 4: 3gat-sa0
Class 5: 3gat-sa1
Class 6: 4gat-sa0
Class 7: 4gat-sa1
Class 8: 9gat-sa0
Class 9: 9gat-sa1
Class 10: 5gat-sa0
```

```
Class 11: 5gat-sa1
Class 12: 6gat-sa0
Class 13: 6gat-sa1
Class 14: 7gat-sa0
Class 15: 7gat-sa1
Class 16: 8gat-sa0
Class 17: 8gat-sa1

[0] Read the input net-list
[1] Perform fault collapsing
[2] List fault classes
[3] Simulate
[4] Generate tests (D-Algorithm)
[5] Generate tests (PODEM)
[6] Generate tests (Boolean Satisfaibility)
[7] Exit

Select an option: 4

Algorithm D results:
Fault 1gat-sa0: test = 11X0
Fault 1gat-sa1: test = 01X0
Fault 2gat-sa0: test = 110X
Fault 2gat-sa1: test = 100X
Fault 3gat-sa0: test = 101X
Fault 3gat-sa1: test = 100X
Fault 4gat-sa0: test = 0XX1
Fault 4gat-sa1: test = 0XX0
Fault 9gat-sa0: test = 11XX
Fault 9gat-sa1: test = 0XX0
Fault 5gat-sa0: test = 11XX
Fault 5gat-sa1: test = 100X
Fault 6gat-sa0: test = 0XX1
Fault 6gat-sa1: test = 1001
Fault 7gat-sa0: test = 11XX
Fault 7gat-sa1: test = 0XX0
Fault 8gat-sa0: test = 0XX1
Fault 8gat-sa1: test = 100X
Detected 18/18 faults
```

```
[0] Read the input net-list
[1] Perform fault collapsing
[2] List fault classes
[3] Simulate
[4] Generate tests (D-Algorithm)
[5] Generate tests (PODEM)
[6] Generate tests (Boolean Satisfaibility)
[7] Exit

Select an option: 5

Algorithm PODEM results:
Fault 1gat-sa0: test = 11X0
Fault 1gat-sa1: test = 01X0
Fault 2gat-sa0: test = 110X
Fault 2gat-sa1: test = 100X
Fault 3gat-sa0: test = 101X
Fault 3gat-sa1: test = 100X
Fault 4gat-sa0: test = 0XX1
Fault 4gat-sa1: test = 0XX0
Fault 9gat-sa0: test = 11XX
Fault 9gat-sa1: test = 0XX0
Fault 5gat-sa0: test = 11XX
Fault 5gat-sa1: test = 100X
Fault 6gat-sa0: test = 0XX1
Fault 6gat-sa1: test = 1001
Fault 7gat-sa0: test = 11XX
Fault 7gat-sa1: test = 0XX0
Fault 8gat-sa0: test = 0XX1
Fault 8gat-sa1: test = 100X
Detected 18/18 faults

[0] Read the input net-list
[1] Perform fault collapsing
[2] List fault classes
[3] Simulate
[4] Generate tests (D-Algorithm)
[5] Generate tests (PODEM)
[6] Generate tests (Boolean Satisfaibility)
[7] Exit
```

```
Select an option: 6

Algorithm SAT results:
Fault 1gat-sa0: test = 1001
Fault 1gat-sa1: test = 0001
Fault 2gat-sa0: test = 1101
Fault 2gat-sa1: test = 1001
Fault 3gat-sa0: test = 1011
Fault 3gat-sa1: test = 1001
Fault 4gat-sa0: test = 0011
Fault 4gat-sa1: test = 0000
Fault 9gat-sa0: test = 1100
Fault 9gat-sa1: test = 0000
Fault 5gat-sa0: test = 1100
Fault 5gat-sa1: test = 1000
Fault 6gat-sa0: test = 0001
Fault 6gat-sa1: test = 1001
Fault 7gat-sa0: test = 1010
Fault 7gat-sa1: test = 1001
Fault 8gat-sa0: test = 0001
Fault 8gat-sa1: test = 0000
Detected 18/18 faults

[0] Read the input net-list
[1] Perform fault collapsing
[2] List fault classes
[3] Simulate
[4] Generate tests (D-Algorithm)
[5] Generate tests (PODEM)
[6] Generate tests (Boolean Satisfaibility)
[7] Exit

Select an option: 3
Enter test vector for PIs ['1gat', '2gat', '3gat', '4gat']: 0000
Enter faults (e.g., a-sa0,b-sa1) or leave blank: 8gat-sa1
Fault 8gat-sa1 detected at outputs ['9gat']: {'9gat': "D'"}

[0] Read the input net-list
[1] Perform fault collapsing
```

```
[2] List fault classes
[3] Simulate
[4] Generate tests (D-Algorithm)
[5] Generate tests (PODEM)
[6] Generate tests (Boolean Satisfaibility)
[7] Exit
```

t4_21.ckt:

```
Select an option: 0
Enter path to .ckt file: examples/t4_21.ckt
Loaded circuit with 10 nodes, 5 PIs, 1 POs.

[0] Read the input net-list
[1] Perform fault collapsing
[2] List fault classes
[3] Simulate
[4] Generate tests (D-Algorithm)
[5] Generate tests (PODEM)
[6] Generate tests (Boolean Satisfaibility)
[7] Exit

Select an option: 1
Created 20 fault classes

[0] Read the input net-list
[1] Perform fault collapsing
[2] List fault classes
[3] Simulate
[4] Generate tests (D-Algorithm)
[5] Generate tests (PODEM)
[6] Generate tests (Boolean Satisfaibility)
[7] Exit

Select an option: 2
Class 0: 1gat-sa0
Class 1: 1gat-sa1
Class 2: 2gat-sa0
Class 3: 2gat-sa1
Class 4: 3gat-sa0
Class 5: 3gat-sa1
```

```
Class 6: 4gat-sa0
Class 7: 4gat-sa1
Class 8: 5gat-sa0
Class 9: 5gat-sa1
Class 10: 9gat-sa0
Class 11: 9gat-sa1
Class 12: 10gat-sa0
Class 13: 10gat-sa1
Class 14: 6gat-sa0
Class 15: 6gat-sa1
Class 16: 7gat-sa0
Class 17: 7gat-sa1
Class 18: 8gat-sa0
Class 19: 8gat-sa1

[0] Read the input net-list
[1] Perform fault collapsing
[2] List fault classes
[3] Simulate
[4] Generate tests (D-Algorithm)
[5] Generate tests (PODEM)
[6] Generate tests (Boolean Satisfaibility)
[7] Exit

Select an option: 4

Algorithm D results:
Fault 1gat-sa0: test = 110X0
Fault 1gat-sa1: test = 010X0
Fault 2gat-sa0: test = 110X0
Fault 2gat-sa1: test = 100X0
Fault 3gat-sa0: test = 0X100
Fault 3gat-sa1: test = 0X000
Fault 4gat-sa0: test = XX110
Fault 4gat-sa1: test = XX100
Fault 5gat-sa0: test = 110X1
Fault 5gat-sa1: test = 110X0
Fault 9gat-sa0: test = 0X0XX
Fault 9gat-sa1: test = XX100
Fault 10gat-sa0: test = 110X0
```

```
Fault 10gat-sa1: test = 0X0X0
Fault 6gat-sa0: test = 0X0X0
Fault 6gat-sa1: test = XX100
Fault 7gat-sa0: test = XX110
Fault 7gat-sa1: test = 110X0
Fault 8gat-sa0: test = XX1X1
Fault 8gat-sa1: test = 110X0
Detected 20/20 faults

[0] Read the input net-list
[1] Perform fault collapsing
[2] List fault classes
[3] Simulate
[4] Generate tests (D-Algorithm)
[5] Generate tests (PODEM)
[6] Generate tests (Boolean Satisfaibility)
[7] Exit

Select an option: 5

Algorithm PODEM results:
Fault 1gat-sa0: test = 110X0
Fault 1gat-sa1: test = 010X0
Fault 2gat-sa0: test = 110X0
Fault 2gat-sa1: test = 100X0
Fault 3gat-sa0: test = 0X100
Fault 3gat-sa1: test = 0X000
Fault 4gat-sa0: test = XX110
Fault 4gat-sa1: test = XX100
Fault 5gat-sa0: test = 110X1
Fault 5gat-sa1: test = 110X0
Fault 9gat-sa0: test = 0X0XX
Fault 9gat-sa1: test = XX100
Fault 10gat-sa0: test = 110X0
Fault 10gat-sa1: test = 0X0X0
Fault 6gat-sa0: test = 0X0X0
Fault 6gat-sa1: test = XX100
Fault 7gat-sa0: test = XX110
Fault 7gat-sa1: test = 110X0
Fault 8gat-sa0: test = XX1X1
```

```
Fault 8gat-sa1: test = 110X0
Detected 20/20 faults

[0] Read the input net-list
[1] Perform fault collapsing
[2] List fault classes
[3] Simulate
[4] Generate tests (D-Algorithm)
[5] Generate tests (PODEM)
[6] Generate tests (Boolean Satisfaibility)
[7] Exit

Select an option: 6

Algorithm SAT results:
Fault 1gat-sa0: test = 11000
Fault 1gat-sa1: test = 01000
Fault 2gat-sa0: test = 11000
Fault 2gat-sa1: test = 10000
Fault 3gat-sa0: test = 11110
Fault 3gat-sa1: test = 01000
Fault 4gat-sa0: test = 10110
Fault 4gat-sa1: test = 01100
Fault 5gat-sa0: test = 11001
Fault 5gat-sa1: test = 11010
Fault 9gat-sa0: test = 10000
Fault 9gat-sa1: test = 11000
Fault 10gat-sa0: test = 11000
Fault 10gat-sa1: test = 00000
Fault 6gat-sa0: test = 01000
Fault 6gat-sa1: test = 11000
Fault 7gat-sa0: test = 11110
Fault 7gat-sa1: test = 11000
Fault 8gat-sa0: test = 11001
Fault 8gat-sa1: test = 11000
Detected 20/20 faults

[0] Read the input net-list
[1] Perform fault collapsing
[2] List fault classes
```

```
[3] Simulate
[4] Generate tests (D-Algorithm)
[5] Generate tests (PODEM)
[6] Generate tests (Boolean Satisfaibility)
[7] Exit

Select an option: 3
Enter test vector for PIs ['1gat', '2gat', '3gat', '4gat', '5gat']: 11000
Enter faults (e.g., a-sa0,b-sa1) or leave blank: 8gat-sa1
Fault 8gat-sa1 detected at outputs ['9gat']: {'9gat': "D'"}

[0] Read the input net-list
[1] Perform fault collapsing
[2] List fault classes
[3] Simulate
[4] Generate tests (D-Algorithm)
[5] Generate tests (PODEM)
[6] Generate tests (Boolean Satisfaibility)
[7] Exit
```

t5_10.ckt:

```
Select an option: 0
Enter path to .ckt file: examples/t5_10.ckt
Loaded circuit with 10 nodes, 5 PIs, 1 POs.

[0] Read the input net-list
[1] Perform fault collapsing
[2] List fault classes
[3] Simulate
[4] Generate tests (D-Algorithm)
[5] Generate tests (PODEM)
[6] Generate tests (Boolean Satisfaibility)
[7] Exit

Select an option: 1
Created 20 fault classes

[0] Read the input net-list
[1] Perform fault collapsing
[2] List fault classes
```

```
[3] Simulate
[4] Generate tests (D-Algorithm)
[5] Generate tests (PODEM)
[6] Generate tests (Boolean Satisfaibility)
[7] Exit

Select an option: 2
Class 0: agat-sa0
Class 1: agat-sa1
Class 2: bgat-sa0
Class 3: bgat-sa1
Class 4: cgat-sa0
Class 5: cgat-sa1
Class 6: dgat-sa0
Class 7: dgat-sa1
Class 8: egat-sa0
Class 9: egat-sa1
Class 10: mgat-sa0
Class 11: mgat-sa1
Class 12: fgat-sa0
Class 13: fgat-sa1
Class 14: jgat-sa0
Class 15: jgat-sa1
Class 16: igat-sa0
Class 17: igat-sa1
Class 18: kgat-sa0
Class 19: kgat-sa1

[0] Read the input net-list
[1] Perform fault collapsing
[2] List fault classes
[3] Simulate
[4] Generate tests (D-Algorithm)
[5] Generate tests (PODEM)
[6] Generate tests (Boolean Satisfaibility)
[7] Exit

Select an option: 4

Algorithm D results:
```

```
Fault agat-sa0: test = 110X0
Fault agat-sa1: test = 010X0
Fault bgat-sa0: test = 110X0
Fault bgat-sa1: test = 100X0
Fault cgat-sa0: test = 0X100
Fault cgat-sa1: test = 0X000
Fault dgat-sa0: test = XX110
Fault dgat-sa1: test = XX100
Fault egat-sa0: test = 110X1
Fault egat-sa1: test = 110X0
Fault mgat-sa0: test = 0X0XX
Fault mgat-sa1: test = 110X0
Fault fgat-sa0: test = 110X0
Fault fgat-sa1: test = 0X0X0
Fault jgat-sa0: test = 0X0X0
Fault jgat-sa1: test = 110X0
Fault igat-sa0: test = XX110
Fault igat-sa1: test = 110X0
Fault kgat-sa0: test = XX11X
Fault kgat-sa1: test = 110X0
Detected 20/20 faults

[0] Read the input net-list
[1] Perform fault collapsing
[2] List fault classes
[3] Simulate
[4] Generate tests (D-Algorithm)
[5] Generate tests (PODEM)
[6] Generate tests (Boolean Satisfaibility)
[7] Exit

Select an option: 5

Algorithm PODEM results:
Fault agat-sa0: test = 110X0
Fault agat-sa1: test = 010X0
Fault bgat-sa0: test = 110X0
Fault bgat-sa1: test = 100X0
Fault cgat-sa0: test = 0X100
Fault cgat-sa1: test = 0X000
```

```
Fault dgat-sa0: test = XX110
Fault dgat-sa1: test = XX100
Fault egat-sa0: test = 110X1
Fault egat-sa1: test = 110X0
Fault mgat-sa0: test = 0X0XX
Fault mgat-sa1: test = 110X0
Fault fgat-sa0: test = 110X0
Fault fgat-sa1: test = 0X0X0
Fault jgat-sa0: test = 0X0X0
Fault jgat-sa1: test = 110X0
Fault igat-sa0: test = XX110
Fault igat-sa1: test = 110X0
Fault kgat-sa0: test = XX11X
Fault kgat-sa1: test = 110X0
Detected 20/20 faults

[0] Read the input net-list
[1] Perform fault collapsing
[2] List fault classes
[3] Simulate
[4] Generate tests (D-Algorithm)
[5] Generate tests (PODEM)
[6] Generate tests (Boolean Satisfaibility)
[7] Exit

Select an option: 6

Algorithm SAT results:
Fault agat-sa0: test = 11000
Fault agat-sa1: test = 01000
Fault bgat-sa0: test = 11000
Fault bgat-sa1: test = 10000
Fault cgat-sa0: test = 11110
Fault cgat-sa1: test = 01000
Fault dgat-sa0: test = 10110
Fault dgat-sa1: test = 01100
Fault egat-sa0: test = 11001
Fault egat-sa1: test = 11010
Fault mgat-sa0: test = 10000
Fault mgat-sa1: test = 11000
```

```
Fault fgat-sa0: test = 11000
Fault fgat-sa1: test = 00000
Fault jgat-sa0: test = 01000
Fault jgat-sa1: test = 11000
Fault igat-sa0: test = 11110
Fault igat-sa1: test = 11000
Fault kgat-sa0: test = 11001
Fault kgat-sa1: test = 11000
Detected 20/20 faults

[0] Read the input net-list
[1] Perform fault collapsing
[2] List fault classes
[3] Simulate
[4] Generate tests (D-Algorithm)
[5] Generate tests (PODEM)
[6] Generate tests (Boolean Satisfaibility)
[7] Exit

Select an option: 3
Enter test vector for PIs ['agat', 'bgat', 'cgat', 'dgat', 'egat']: 11000
Enter faults (e.g., a-sa0,b-sa1) or leave blank: kgat-sa1
Fault kgat-sa1 detected at outputs ['mgat']: {'mgat': "D'"}

[0] Read the input net-list
[1] Perform fault collapsing
[2] List fault classes
[3] Simulate
[4] Generate tests (D-Algorithm)
[5] Generate tests (PODEM)
[6] Generate tests (Boolean Satisfaibility)
[7] Exit
```

t5_26a.ckt:

```
Select an option: 0
Enter path to .ckt file: examples/t5_26a.ckt
Loaded circuit with 12 nodes, 3 PIs, 2 POs.

[0] Read the input net-list
[1] Perform fault collapsing
```

```
[2] List fault classes
[3] Simulate
[4] Generate tests (D-Algorithm)
[5] Generate tests (PODEM)
[6] Generate tests (Boolean Satisfaibility)
[7] Exit

Select an option: 1
Created 24 fault classes

[0] Read the input net-list
[1] Perform fault collapsing
[2] List fault classes
[3] Simulate
[4] Generate tests (D-Algorithm)
[5] Generate tests (PODEM)
[6] Generate tests (Boolean Satisfaibility)
[7] Exit

Select an option: 2
Class 0: X-sa0
Class 1: X-sa1
Class 2: Y-sa0
Class 3: Y-sa1
Class 4: CI-sa0
Class 5: CI-sa1
Class 6: S-sa0
Class 7: S-sa1
Class 8: CO-sa0
Class 9: CO-sa1
Class 10: L-sa0
Class 11: L-sa1
Class 12: Q-sa0
Class 13: Q-sa1
Class 14: R-sa0
Class 15: R-sa1
Class 16: N-sa0
Class 17: N-sa1
Class 18: T-sa0
Class 19: T-sa1
```

```
Class 20: U-sa0
Class 21: U-sa1
Class 22: V-sa0
Class 23: V-sa1

[0] Read the input net-list
[1] Perform fault collapsing
[2] List fault classes
[3] Simulate
[4] Generate tests (D-Algorithm)
[5] Generate tests (PODEM)
[6] Generate tests (Boolean Satisfaibility)
[7] Exit

Select an option: 4

Algorithm D results:
Fault X-sa0: test = 111
Fault X-sa1: test = 011
Fault Y-sa0: test = 111
Fault Y-sa1: test = 101
Fault CI-sa0: test = 101
Fault CI-sa1: test = 100
Fault S-sa0: test = 001
Fault S-sa1: test = 000
Fault CO-sa0: test = 11X
Fault CO-sa1: test = 0X0
Fault L-sa0: test = 011
Fault L-sa1: test = 111
Fault Q-sa0: test = 001
Fault Q-sa1: test = 101
Fault R-sa0: test = 111
Fault R-sa1: test = 011
Fault N-sa0: test = 101
Fault N-sa1: test = 001
Fault T-sa0: test = 100
Fault T-sa1: test = 101
Fault U-sa0: test = 000
Fault U-sa1: test = 001
Fault V-sa0: test = 101
```

```
Fault V-sa1: test = 100
Detected 24/24 faults

[0] Read the input net-list
[1] Perform fault collapsing
[2] List fault classes
[3] Simulate
[4] Generate tests (D-Algorithm)
[5] Generate tests (PODEM)
[6] Generate tests (Boolean Satisfaibility)
[7] Exit

Select an option: 5

Algorithm PODEM results:
Fault X-sa0: test = 111
Fault X-sa1: test = 011
Fault Y-sa0: test = 111
Fault Y-sa1: test = 101
Fault CI-sa0: test = 101
Fault CI-sa1: test = 100
Fault S-sa0: test = 001
Fault S-sa1: test = 000
Fault CO-sa0: test = 11X
Fault CO-sa1: test = 0X0
Fault L-sa0: test = 011
Fault L-sa1: test = 111
Fault Q-sa0: test = 001
Fault Q-sa1: test = 101
Fault R-sa0: test = 111
Fault R-sa1: test = 011
Fault N-sa0: test = 101
Fault N-sa1: test = 001
Fault T-sa0: test = 100
Fault T-sa1: test = 101
Fault U-sa0: test = 000
Fault U-sa1: test = 001
Fault V-sa0: test = 101
Fault V-sa1: test = 100
Detected 24/24 faults
```

```
[0] Read the input net-list
[1] Perform fault collapsing
[2] List fault classes
[3] Simulate
[4] Generate tests (D-Algorithm)
[5] Generate tests (PODEM)
[6] Generate tests (Boolean Satisfaibility)
[7] Exit

Select an option: 6

Algorithm SAT results:
Fault X-sa0: test = 110
Fault X-sa1: test = 010
Fault Y-sa0: test = 110
Fault Y-sa1: test = 100
Fault CI-sa0: test = 111
Fault CI-sa1: test = 110
Fault S-sa0: test = 010
Fault S-sa1: test = 011
Fault CO-sa0: test = 011
Fault CO-sa1: test = 000
Fault L-sa0: test = 000
Fault L-sa1: test = 110
Fault Q-sa0: test = 000
Fault Q-sa1: test = 101
Fault R-sa0: test = 000
Fault R-sa1: test = 011
Fault N-sa0: test = 011
Fault N-sa1: test = 110
Fault T-sa0: test = 000
Fault T-sa1: test = 011
Fault U-sa0: test = 101
Fault U-sa1: test = 111
Fault V-sa0: test = 101
Fault V-sa1: test = 010
Detected 24/24 faults

[0] Read the input net-list
```

```
[1] Perform fault collapsing
[2] List fault classes
[3] Simulate
[4] Generate tests (D-Algorithm)
[5] Generate tests (PODEM)
[6] Generate tests (Boolean Satisfaibility)
[7] Exit

Select an option: 3
Enter test vector for PIs ['X', 'Y', 'CI']: 010
Enter faults (e.g., a-sa0,b-sa1) or leave blank: V-sa1
Fault V-sa1 detected at outputs ['S']: {'S': 'D', 'CO': '0'}

[0] Read the input net-list
[1] Perform fault collapsing
[2] List fault classes
[3] Simulate
[4] Generate tests (D-Algorithm)
[5] Generate tests (PODEM)
[6] Generate tests (Boolean Satisfaibility)
[7] Exit
```

t6_24_v1.ckt

```
Select an option: 0
Enter path to .ckt file: examples/t6_24_v1.ckt
Loaded circuit with 22 nodes, 6 PIs, 1 POs.

[0] Read the input net-list
[1] Perform fault collapsing
[2] List fault classes
[3] Simulate
[4] Generate tests (D-Algorithm)
[5] Generate tests (PODEM)
[6] Generate tests (Boolean Satisfaibility)
[7] Exit

Select an option: 1
Created 44 fault classes

[0] Read the input net-list
```

```
[1] Perform fault collapsing
[2] List fault classes
[3] Simulate
[4] Generate tests (D-Algorithm)
[5] Generate tests (PODEM)
[6] Generate tests (Boolean Satisfaibility)
[7] Exit

Select an option: 2
Class 0: 1gat-sa0
Class 1: 1gat-sa1
Class 2: 2gat-sa0
Class 3: 2gat-sa1
Class 4: 3gat-sa0
Class 5: 3gat-sa1
Class 6: 4gat-sa0
Class 7: 4gat-sa1
Class 8: 5gat-sa0
Class 9: 5gat-sa1
Class 10: 6gat-sa0
Class 11: 6gat-sa1
Class 12: 16gat-sa0
Class 13: 16gat-sa1
Class 14: 13gat-sa0
Class 15: 13gat-sa1
Class 16: 17gat-sa0
Class 17: 17gat-sa1
Class 18: 8gat-sa0
Class 19: 8gat-sa1
Class 20: 12gat-sa0
Class 21: 12gat-sa1
Class 22: 11gat-sa0
Class 23: 11gat-sa1
Class 24: 7gat-sa0
Class 25: 7gat-sa1
Class 26: 10gat-sa0
Class 27: 10gat-sa1
Class 28: 18gat-sa0
Class 29: 18gat-sa1
Class 30: 9gat-sa0
```

```
Class 31: 9gat-sa1
Class 32: 14gat-sa0
Class 33: 14gat-sa1
Class 34: 15gat-sa0
Class 35: 15gat-sa1
Class 36: 19gat-sa0
Class 37: 19gat-sa1
Class 38: 20gat-sa0
Class 39: 20gat-sa1
Class 40: 21gat-sa0
Class 41: 21gat-sa1
Class 42: 22gat-sa0
Class 43: 22gat-sa1

[0] Read the input net-list
[1] Perform fault collapsing
[2] List fault classes
[3] Simulate
[4] Generate tests (D-Algorithm)
[5] Generate tests (PODEM)
[6] Generate tests (Boolean Satisfaibility)
[7] Exit

Select an option: 4

Algorithm D results:
Fault 1gat-sa0: test = 100000
Fault 1gat-sa1: test = 000000
Fault 2gat-sa0: test = 010000
Fault 2gat-sa1: test = 000000
Fault 3gat-sa0: test = 111111
Fault 3gat-sa1: test = 110111
Fault 4gat-sa0: test = 111111
Fault 4gat-sa1: test = 111011
Fault 5gat-sa0: test = 111111
Fault 5gat-sa1: test = 111101
Fault 6gat-sa0: test = 111111
Fault 6gat-sa1: test = 111110
Fault 16gat-sa0: test = XX1XX0
Fault 16gat-sa1: test = 000000
```

```
Fault 13gat-sa0: test = 000000
Fault 13gat-sa1: test = 0X0010
Fault 17gat-sa0: test = 0XX111
Fault 17gat-sa1: test = 111111
Fault 8gat-sa0: test = 010000
Fault 8gat-sa1: test = 111111
Fault 12gat-sa0: test = 000000
Fault 12gat-sa1: test = 010000
Fault 11gat-sa0: test = 000000
Fault 11gat-sa1: test = 000100
Fault 7gat-sa0: test = 100000
Fault 7gat-sa1: test = 111111
Fault 10gat-sa0: test = 000000
Fault 10gat-sa1: test = 100000
Fault 18gat-sa0: test = 111111
Fault 18gat-sa1: test = 10X111
Fault 9gat-sa0: test = 001000
Fault 9gat-sa1: test = 111111
Fault 14gat-sa0: test = 000000
Fault 14gat-sa1: test = 001000
Fault 15gat-sa0: test = 000000
Fault 15gat-sa1: test = 00X001
Fault 19gat-sa0: test = 000000
Fault 19gat-sa1: test = 100000
Fault 20gat-sa0: test = 000000
Fault 20gat-sa1: test = 010000
Fault 21gat-sa0: test = 000000
Fault 21gat-sa1: test = 001000
Fault 22gat-sa0: test = 000000
Fault 22gat-sa1: test = 1X00X0
Detected 44/44 faults

[0] Read the input net-list
[1] Perform fault collapsing
[2] List fault classes
[3] Simulate
[4] Generate tests (D-Algorithm)
[5] Generate tests (PODEM)
[6] Generate tests (Boolean Satisfaibility)
[7] Exit
```

Select an option: 5

Algorithm PODEM results:
Fault 1gat-sa0: test = 100000
Fault 1gat-sa1: test = 000000
Fault 2gat-sa0: test = 010000
Fault 2gat-sa1: test = 000000
Fault 3gat-sa0: test = 111111
Fault 3gat-sa1: test = 110111
Fault 4gat-sa0: test = 111111
Fault 4gat-sa1: test = 111011
Fault 5gat-sa0: test = 111111
Fault 5gat-sa1: test = 111101
Fault 6gat-sa0: test = 111111
Fault 6gat-sa1: test = 111110
Fault 16gat-sa0: test = XX1XX0
Fault 16gat-sa1: test = 000000
Fault 13gat-sa0: test = 000000
Fault 13gat-sa1: test = 0X0010
Fault 17gat-sa0: test = 0XX111
Fault 17gat-sa1: test = 111111
Fault 8gat-sa0: test = 010000
Fault 8gat-sa1: test = 111111
Fault 12gat-sa0: test = 000000
Fault 12gat-sa1: test = 010000
Fault 11gat-sa0: test = 000000
Fault 11gat-sa1: test = 000100
Fault 7gat-sa0: test = 100000
Fault 7gat-sa1: test = 111111
Fault 10gat-sa0: test = 000000
Fault 10gat-sa1: test = 100000
Fault 18gat-sa0: test = 111111
Fault 18gat-sa1: test = 10X111
Fault 9gat-sa0: test = 001000
Fault 9gat-sa1: test = 111111
Fault 14gat-sa0: test = 000000
Fault 14gat-sa1: test = 001000
Fault 15gat-sa0: test = 000000
Fault 15gat-sa1: test = 00X001

```
Fault 19gat-sa0: test = 000000
Fault 19gat-sa1: test = 100000
Fault 20gat-sa0: test = 000000
Fault 20gat-sa1: test = 010000
Fault 21gat-sa0: test = 000000
Fault 21gat-sa1: test = 001000
Fault 22gat-sa0: test = 000000
Fault 22gat-sa1: test = 1X00X0
Detected 44/44 faults

[0] Read the input net-list
[1] Perform fault collapsing
[2] List fault classes
[3] Simulate
[4] Generate tests (D-Algorithm)
[5] Generate tests (PODEM)
[6] Generate tests (Boolean Satisfaibility)
[7] Exit

Select an option: 6

Algorithm SAT results:
Fault 1gat-sa0: test = 100000
Fault 1gat-sa1: test = 000000
Fault 2gat-sa0: test = 010000
Fault 2gat-sa1: test = 000000
Fault 3gat-sa0: test = 001000
Fault 3gat-sa1: test = 000000
Fault 4gat-sa0: test = 000100
Fault 4gat-sa1: test = 000000
Fault 5gat-sa0: test = 000010
Fault 5gat-sa1: test = 000000
Fault 6gat-sa0: test = 000001
Fault 6gat-sa1: test = 111110
Fault 16gat-sa0: test = 111110
Fault 16gat-sa1: test = 111111
Fault 13gat-sa0: test = 111111
Fault 13gat-sa1: test = 010010
Fault 17gat-sa0: test = 000110
Fault 17gat-sa1: test = 111111
```

```
Fault 8gat-sa0: test = 111101
Fault 8gat-sa1: test = 111111
Fault 12gat-sa0: test = 111111
Fault 12gat-sa1: test = 010000
Fault 11gat-sa0: test = 111111
Fault 11gat-sa1: test = 000100
Fault 7gat-sa0: test = 111011
Fault 7gat-sa1: test = 111111
Fault 10gat-sa0: test = 000000
Fault 10gat-sa1: test = 111011
Fault 18gat-sa0: test = 111111
Fault 18gat-sa1: test = 100100
Fault 9gat-sa0: test = 111110
Fault 9gat-sa1: test = 111111
Fault 14gat-sa0: test = 000000
Fault 14gat-sa1: test = 111110
Fault 15gat-sa0: test = 111111
Fault 15gat-sa1: test = 000001
Fault 19gat-sa0: test = 000000
Fault 19gat-sa1: test = 100100
Fault 20gat-sa0: test = 000000
Fault 20gat-sa1: test = 000010
Fault 21gat-sa0: test = 000000
Fault 21gat-sa1: test = 111110
Fault 22gat-sa0: test = 111111
Fault 22gat-sa1: test = 100110
Detected 44/44 faults

[0] Read the input net-list
[1] Perform fault collapsing
[2] List fault classes
[3] Simulate
[4] Generate tests (D-Algorithm)
[5] Generate tests (PODEM)
[6] Generate tests (Boolean Satisfaibility)
[7] Exit

Select an option: 3
Enter test vector for PIs ['1gat', '2gat', '3gat', '4gat', '5gat', '6gat']: 100110
Enter faults (e.g., a-sa0,b-sa1) or leave blank: 22gat-sa1
```

Fault 22gat-sa1 detected at outputs ['16gat']: {'16gat': 'D'}

[0] Read the input net-list
[1] Perform fault collapsing
[2] List fault classes
[3] Simulate
[4] Generate tests (D-Algorithm)
[5] Generate tests (PODEM)
[6] Generate tests (Boolean Satisfaibility)
[7] Exit

Select an option: 7

Conclusion

This project provides a complete ATPG environment combining classical structural algorithms (D-Algorithm and PODEM) with a modern SAT-based formulation. The implementation successfully parses all benchmark circuits, handles flexible .ckt formats, performs correct five-valued simulation, and generates test vectors that activate and propagate every stuck-at fault in the tested circuits. The cross-validation between three independent ATPG engines, along with explicit simulation of each returned vector, confirms both the correctness and robustness of the tool.

The resulting framework is suitable for coursework, research, and further extensions such as SCOAP controllability/observability metrics, redundancy identification, sequential ATPG, or optimized SAT encodings.